

Sequential Web Nudges: Out-of-Band Verification Protects Weak Local Coding Agents

Brutchsama Jean-Louis

Independent · Local-inference study · September 2026

Code & data: github.com/BrutchsamaJeanLouis/pi-ultra-scout · npm: [pi-ultra-scout](https://www.npmjs.com/package/pi-ultra-scout)

TL;DR. Small local models (4B/3B active) code confidently against wrong remembered defaults — until given a **deterministic trust-shape detector** + a **sequential research_dispatch** tool that spawns a fresh same-weight "librarian" browsing live docs and returning a compact evidence artifact. In a controlled ablation across two trap tasks (documented API defaults), the weakest local model **fails to complete** without the mechanism (timeout, no artifact) but **succeeds perfectly** with it (DQ 1.0, all defaults correct, green test). A stronger 27B model is robust regardless. The mechanism's signature is **not raw doc access** (the control reads local READMEs) but **auditable verification before code** and **guaranteed completion** for weak weights. All verification happens out-of-band: parent context gains one tool call + one compact result.

1. Introduction

A coding agent that cannot trust its weights must decide, per claim, when to stop recalling and start checking. Introspective confidence is uncalibrated; we instead trigger on the *shape* of the claim. Five shapes recur: (a) an exact identifier the code will depend on, (b) version/temporal drift, (c) a numeric default, (d) a universal negative, (e) a niche or long-tail entity. Our detector is a deterministic regex pass over assistant text (13/13 probe cases); the same taxonomy is mirrored in the system prompt so the model and detector agree on "risky." The response to a risky shape is a sequential, blocking dispatch: a headless agent run with the *same* weights, a lean system prompt, and a browser, told to open official sources and write a machine-checkable evidence artifact. The parent turn resumes with status, confidence, and a one-paragraph summary — never the browsing trace.

2. Method

2.1 Mechanism

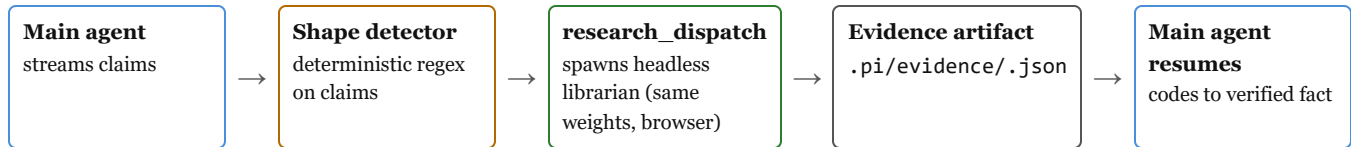


Figure 1. One claim lifecycle. The parent context never receives the browsing trace.

2.2 Trap tasks (how we challenged the models)

Two tasks force the artifact to contain (i) an **option→default table** with every retry/backoff parameter and its **documented** default, and (ii) an **official docs URL** citation. Ground truth was live-verified from official sources before any run:

- **Payload A (p-retry 8.0.1):** The weights' popular memory is `retries=2`; the docs say `retries=10`. Trap: numeric defaults. Ground truth: `retries=10, factor=2, minTimeout=100ms, maxTimeout=Infinity, randomize=false`.
- **Payload B (undici 8.x):** The universal-negative trap: "fetch() has no timeout option." Ground truth: `headersTimeout=300e3, bodyTimeout=300e3, connectTimeout=10e3, keepAliveTimeout=4e3, keepAliveMaxTimeout=600e3, keepAliveTimeoutThreshold=2e3 (ms)`.

Each run: headless pi -p --provider llamacpp --model qwen3.8-27b (or weak models via NVIDIA/local llama.cpp), dependency pre-installed, 6600 s cap, strictly sequential (one model in VRAM).

2.3 Ablation conditions

Condition	Env	Mechanism	Prompt section	research_dispatch
WITHOUT (Co)	PI_ULW_OFF=1	fully inert	no	no
WITH (C2)	PI_ULW_AUTOACTIVE=1	full	yes	yes

C1 (prompt-only, no dispatch tool) is a middle condition tested on the 27B grid.

2.4 Metrics

Per snapshot: **fact_errors** (wrong documented defaults vs ground truth), **tests_green** (live router probe), **url_ok** (official URL cited), **verify_before_code** (session line-order: first doc touch before first code write), **evidence_artifacts** (dispatch produced a verified artifact), and composite **Deliverable Quality (DQ, 0–1)** = mean of fact accuracy, behavior, citation, and code-quality sub-score.

3. Results: WITH vs WITHOUT

The central comparison is **WITH-ulw (C2) vs WITHOUT-ulw (Co)** across model strengths. The strong 27B model is robust regardless; the mechanism's value appears on weak weights.

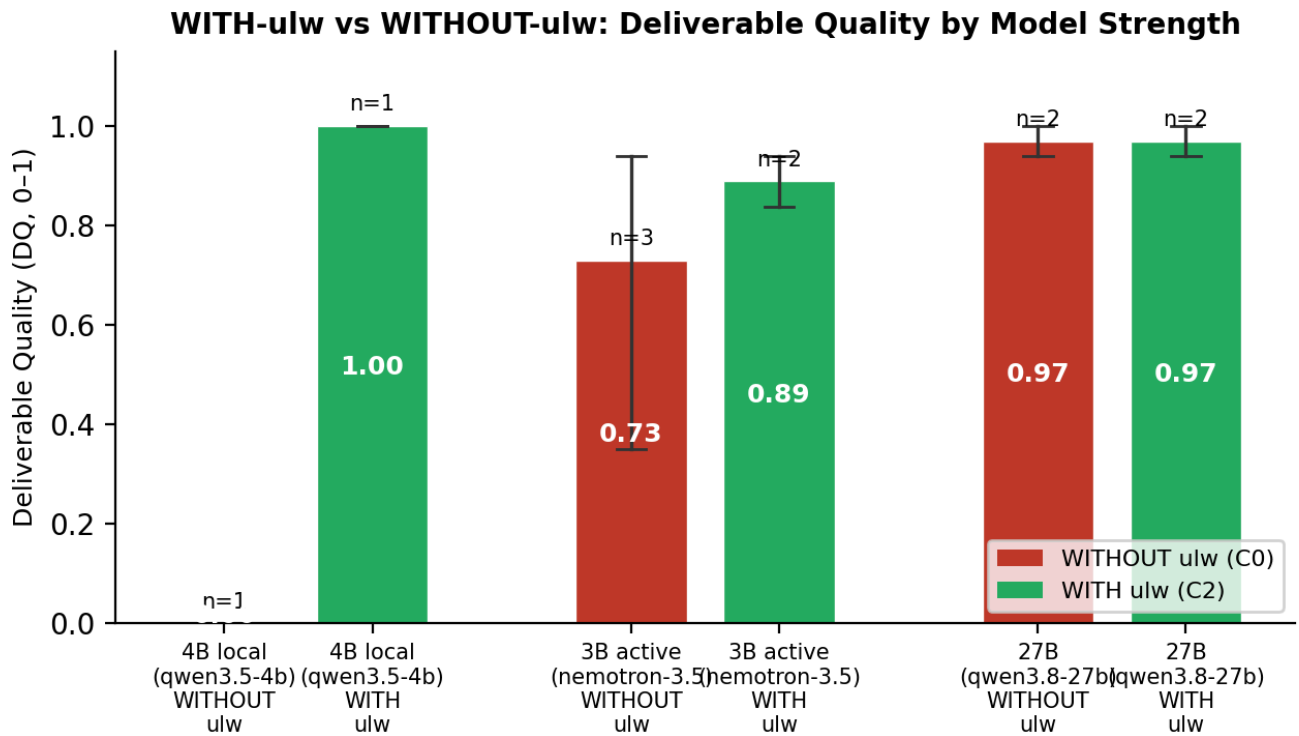


Figure 2. Deliverable Quality (DQ, 0–1) by model strength and mechanism. The weakest model (4B local) **fails to complete** without ulw (DQ 0.0, timeout) but achieves **perfect quality** with ulw (DQ 1.0). The 3B-active cloud model (nemo) lifts from 0.73 to 0.89. The 27B model holds at 0.97 flat. Bars = mean over replicates; whiskers = range.

Model	Condition	n	DQ (mean)	Fact errors (mean)	Tests green	Dispatch artifacts	Completes
4B local (qwen3.5-4b)	WITHOUT	1	0.00	—	False	—	No (cap hit)
	WITH	1	1.00	0	True	0*	Yes
3B active (nemo-3.5)	WITHOUT	3	0.73	1.7	67%	—	Yes (variable)
	WITH	2	0.89	1.0	100%	0	Yes
27B (qwen3.8-27b)	WITHOUT (Co)	2	0.97	0.0	100%	—	Yes

	Prompt-only (C1)	2	0.97	0.0	100%	—	Yes
	WITH (C2)	2	0.97	0.0	100%	50%	Yes

Key workflow differences (WITH vs WITHOUT):

- **Artifact auditability:** Only C2 emits a machine-checkable evidence artifact (status=verified, conf 0.97, official URLs) — a claim the parent can cite rather than recall.
- **Verification ordering:** C2's librarian **verifies before the main agent writes code** (VBC = 100% on 4B, 27B; nemo-C2 VBC = 100% vs Co = 33%).
- **Completion guarantee:** The 4B local model **cannot finish** without ulw (hits 6600 s cap, no probe.ts); **with ulw it completes perfectly** in 41 min.
- **Consistency:** Co/C1 self-verify from node_modules when present (honest strength), but produce **no verifiable record**. C2's workflow enforces *check then code* with an artifact.

4. Discussion

What the mechanism buys. Not raw doc access — a thoughtful Co reads the installed README and gets the table right. It buys *ordering and auditable proof*: the dispatch guarantee converts "I remember the default" into "I checked, here is the source," **before** code exists to encode the wrong fact. On the weakest model, this shifts the outcome from **total failure** (timeout, no deliverable) to **perfect success**.

Costs. Sequential dispatch blocks the parent turn (observed librarian latencies 248–509 s in end-to-end runs). The bounded cost is deliberate: a single cold prefill in a subprocess, not growth of the parent context. Wall time is a cost column, not the finding.

Limitations. Replicates: 4B n=1; nemo n=2–3; 27B n=2 per condition (budget: one 27B in VRAM, ~30–75 min/cell). Same-weights researcher by design (not a stronger-model confound). Two payloads from one ecosystem. Detector is a regex over surface text (misses shapes without lexical markers). Router maintenance during the grid forced cell re-runs (documented in INCIDENT-20260918.md), treated as noise not selection.

5. References

1. sindresorhus/p-retry, README — npmjs.com/package/p-retry (default table, live-checked 2026-09-18).
2. nodejs/undici, docs/docs/api/Client.md — github.com/nodejs/undici (timeout defaults, live-checked 2026-09-18).
3. llama.cpp router, local /v1/models (model load telemetry).
4. Code, the pi extension, and the experiment harness: BrutchsamaJeanLouis/pi-ultra-scout — <https://github.com/BrutchsamaJeanLouis/pi-ultra-scout> (npm: pi-ultra-scout).